**Cairo University**

**Faculty of Computers and Artificial Intelligence**

# CS251

# Introduction to Software Engineering

## $penderman

## Software Design Specifications

Version 1.0

Apr-2026

CS251: Only three
Project: $penderman

# Software Design Specification

## Contents

Instructions [To be removed]

Team

Document Purpose and Audience

System Models

I. Architecture Diagram

II. Class Diagram(s)

III. Class Descriptions

IV. Sequence diagrams

V. State Diagram

VI.ER Diagrams

VII.Schema Diagram

VIII. SOLID Principles

IX. Design Patterns

X. UI design snapshots

Tools

Ownership Report

CS251: Only three
Project: $penderman

# Software Design Specification

## Team

| ID | Name | Email | Mobile |
|---|---|---|---|
| 20240038 | Ahmed Amr Abdelbadea Ali Shawareb (Leader) | 20240038@stud.fci-cu.edu.eg | 01143494177 |
| 20240049 | Ahmed Mohamed Salah El-Din | 20240049@stud.fci-cu.edu.eg | 01096658302 |
| 20240476 | Mohamed Ayser Mohamed El-Hosiny | 20240476@stud.fci-cu.edu.eg | 01026553566 |

## Document Purpose and Audience

**Document Purpose and Audience**

---

**Document Purpose**

This Software Design Specification (SDS) serves as a comprehensive **technical blueprint** for the **$penderman** personal budgeting application. It translates functional requirements into a structured architectural design to guide the development and testing phases.

**Key objectives of this document include:**

- **Defining System Scope:** Outlining the boundaries of the application and its primary interactions with the "System User".
- **Structural Architecture:** Detailing the high-level technical containers, including the **JavaFX GUI**, the **Java Back-end**, and the **Microsoft SQL Server** database.
- **Behavioral Logic:** Providing step-by-step logic for critical operations such as user registration, transaction processing, and financial goal creation.
- **Data Integrity:** Establishing how financial data is persisted and retrieved through secure communication between the back-end and database layers.

# Software Design Specification

This document is intended for stakeholders involved in the technical execution and management of the project:

| Audience | Responsibility |
|---|---|
| **Software Developers** | To implement the specific classes, methods, and logic flows defined in the sequence diagrams. |
| **Project Managers** | To verify that the technical design aligns with the project's scope and business goals. |
| **Software Architects** | To ensure the structural integrity of the container-level interactions and technology stack. |
| **Quality Assurance (QA)** | To design test cases based on the documented logic for transactions and user management. |
| **Database Administrators** | To manage the SQL Server schema required to support the application's data persistence. |

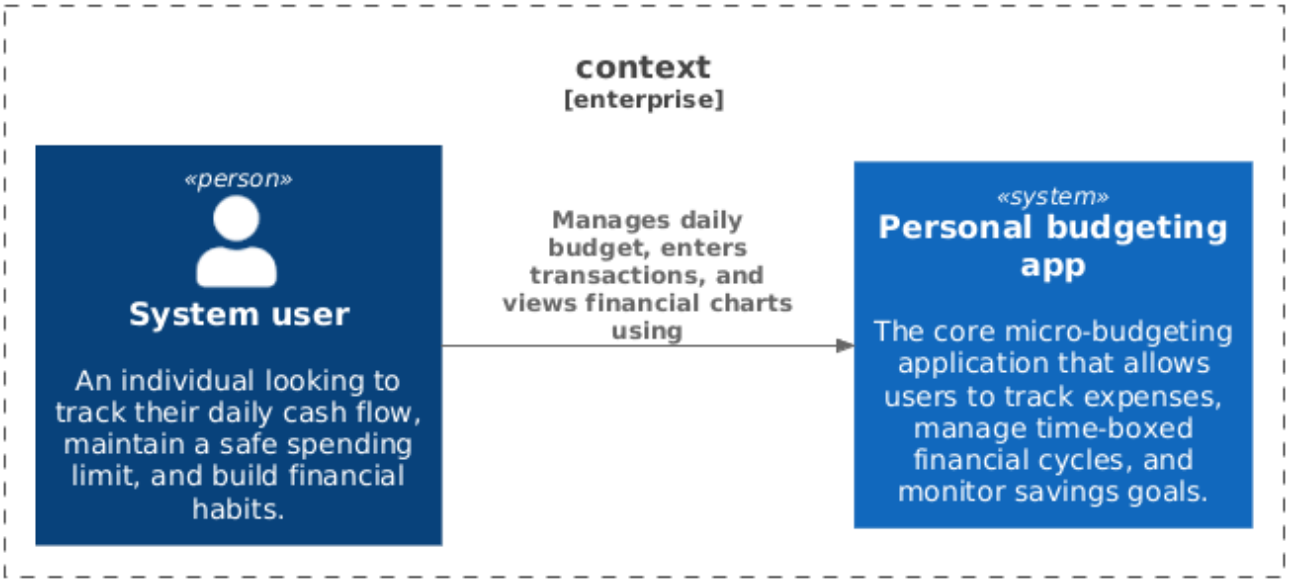
CS251: Only three
Project: $penderman

# Software Design Specification

## System Models

### Architecture Diagram

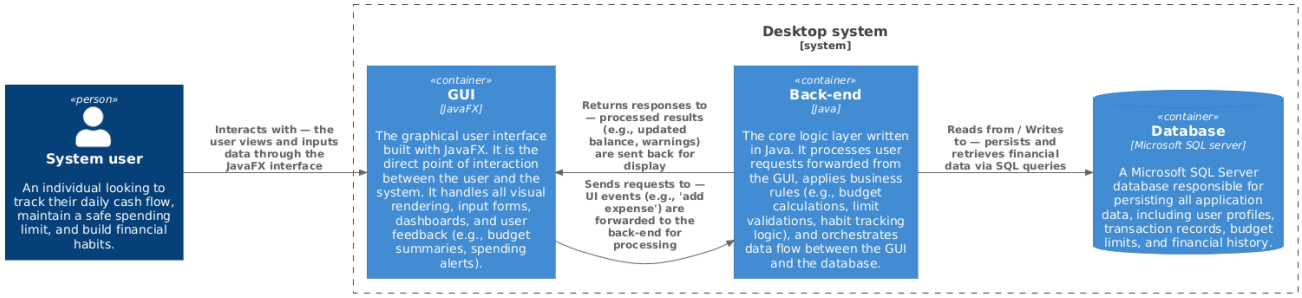
#### 1.Context Diagram



CS251: Only three
Project: $penderman

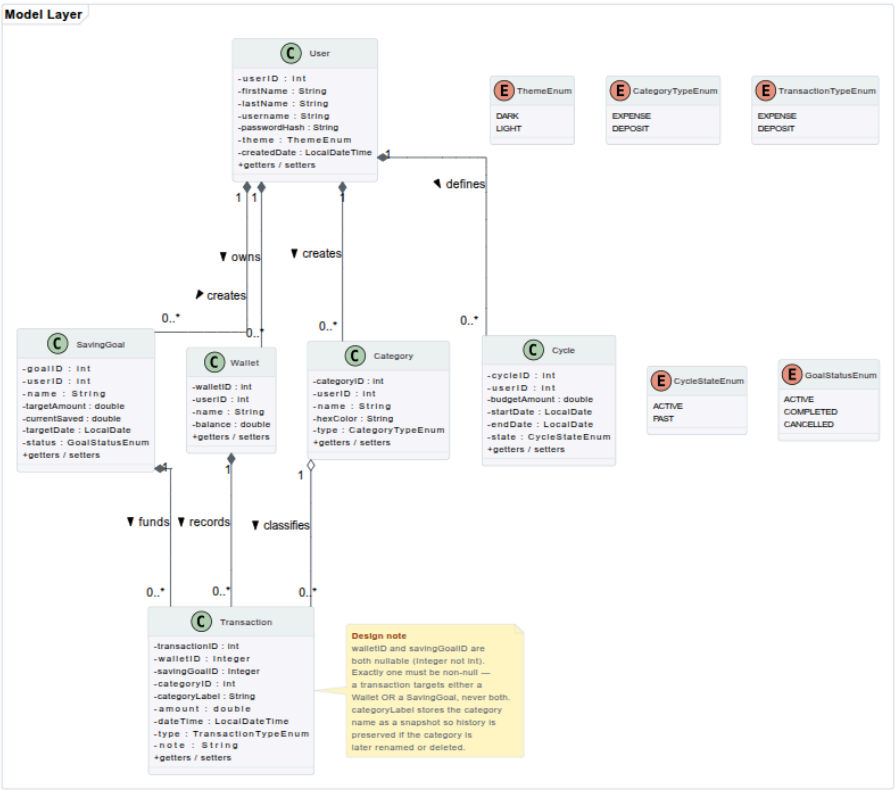
# Software Design Specification

## 2.Container Diagram



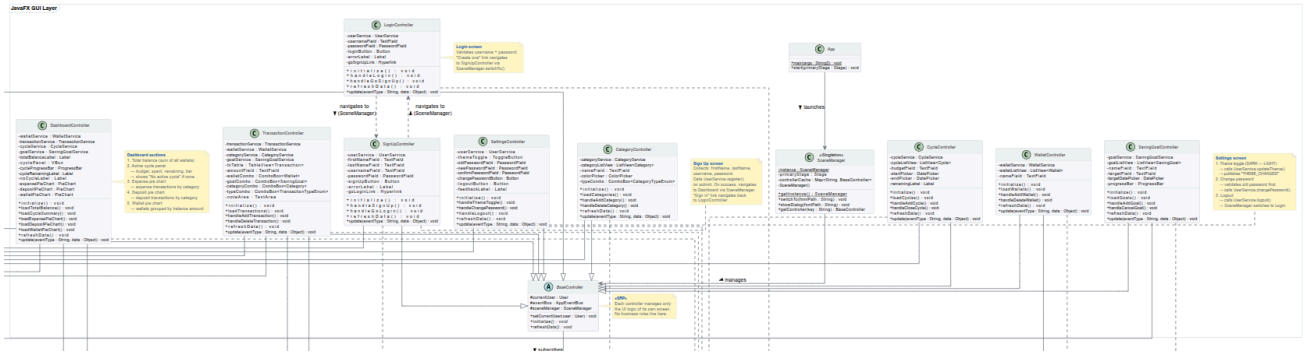

---

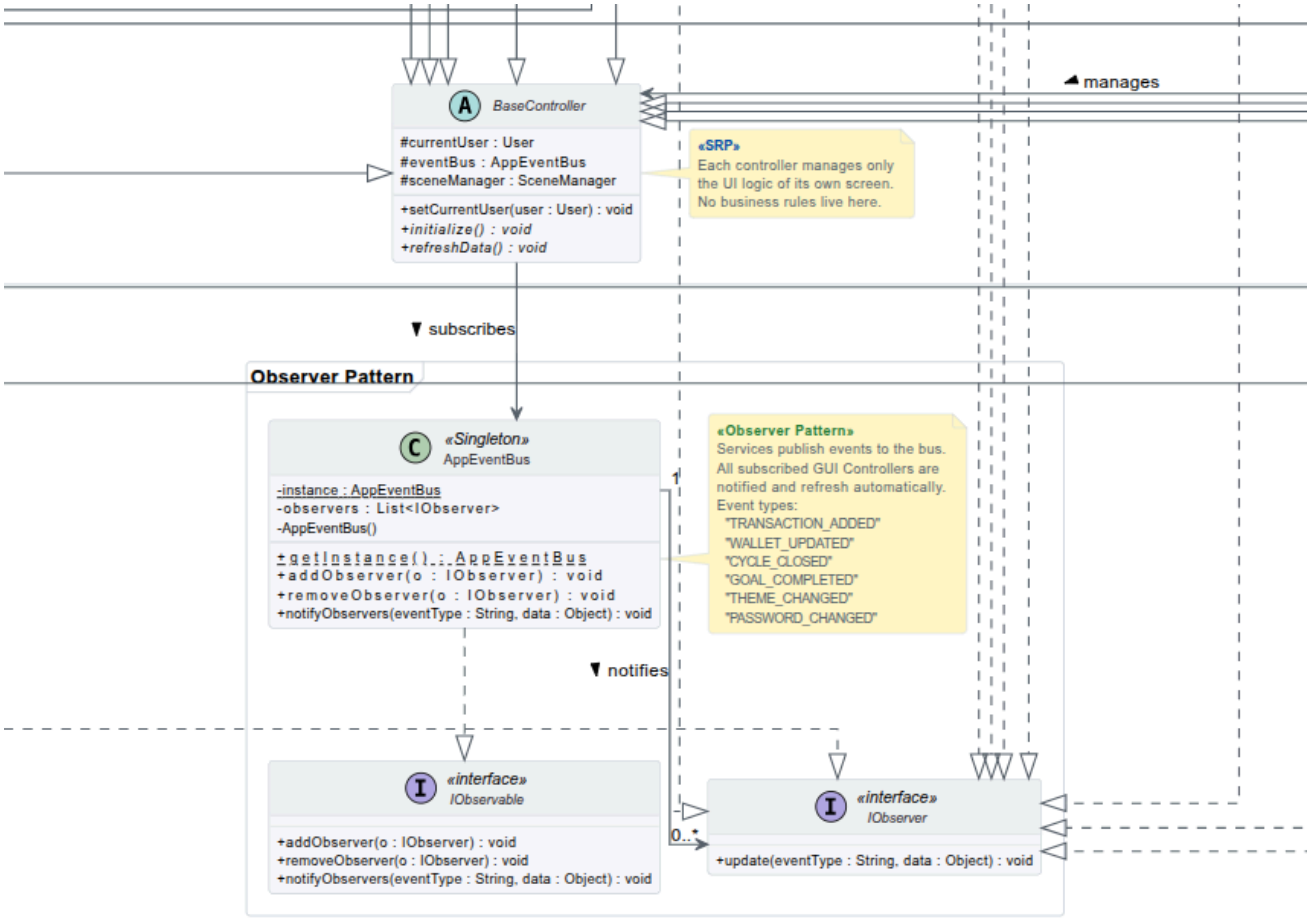
## Class Diagram(s)



CS251: Only three
Project: $penderman

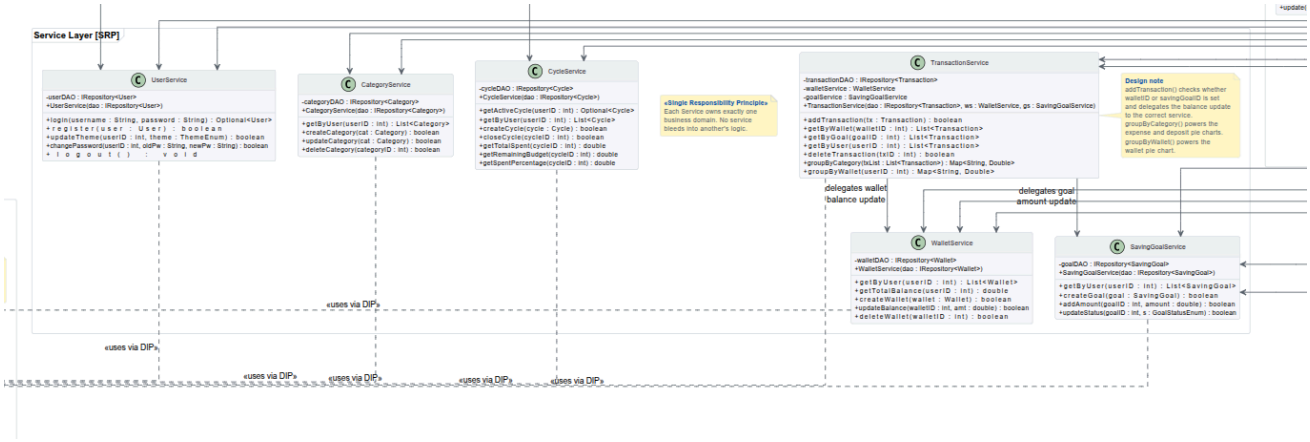
# Software Design Specification

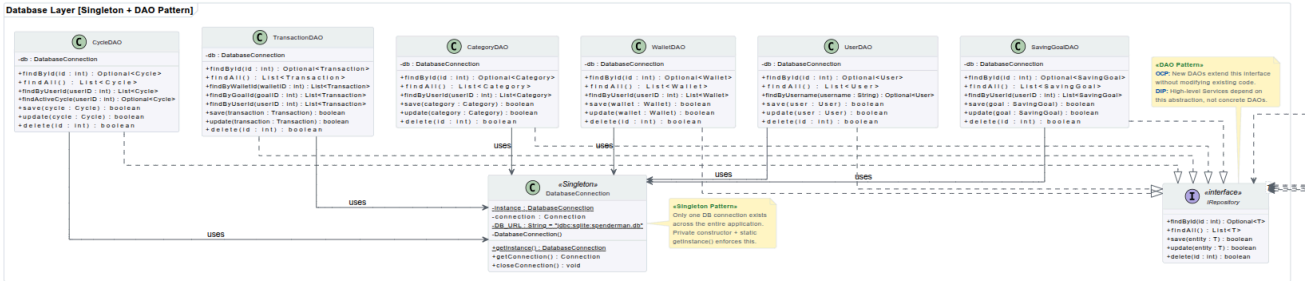




CS251: Only three
Project: $penderman

# Software Design Specification




## For More details and high quality

uml

# Software Design Specification

## Class Descriptions

| Class Name | Description |
|---|---|
| **User** | The root owner of all system entities. Stores authentication details (username, hashed password) and user preferences (UI theme). |
| **Wallet** | Represents a financial account (e.g., "Cash"). Maintains a balance derived entirely from its recorded transactions. |
| **Category** | A user-defined label (e.g., "Food") with an assigned hex color for chart rendering. Distinguishes between expenses and deposits. |
| **Transaction** | The immutable ledger entry. Enforces mutual exclusivity: it must target exactly one Wallet OR one SavingGoal. Stores a static snapshot of the category name. |
| **Cycle** | A time-boxed budget period. Used by the system to calculate the safe daily spending limit over a specific date range. |
| **SavingGoal** | A financial milestone tracking saved amounts against a target. Automatically completes when fully funded. |
| **ThemeEnum** | Defines DARK or LIGHT UI preferences for persistent styling. |
| **CategoryTypeEnum** | Classifies categories as EXPENSE or DEPOSIT for UI filtering. |
| **TransactionTypeEnum** | Determines if a transaction adds (DEPOSIT) or subtracts (EXPENSE) from a balance. |
| **CycleStateEnum** | Tracks if a cycle is ACTIVE or PAST, ensuring only one active budget exists per user. |

# Software Design Specification

| | |
|---|---|
| **GoalStatusEnum** | Tracks milestone progress states: ACTIVE, COMPLETED, or CANCELLED. |
| **DatabaseConnection** | Provides a single, shared JDBC database connection globally, preventing performance overhead from multiple open instances. |
| **IRepository<T>** | Generic interface defining standard CRUD operations (findById, save, delete, etc.) to enable the Dependency Inversion Principle. |
| **UserDAO** | Handles user SQL queries, specifically managing findByUsername() for secure authentication workflows. |
| **WalletDAO** | Handles wallet SQL queries, executing frequent update() commands to adjust balances when transactions occur. |
| **CategoryDAO** | Handles category SQL queries, mapping UI hex colors and Enum types to the database schema. |
| **TransactionDAO** | Handles complex transaction queries. Explicitly manages the nullable walletID and savingGoalID foreign keys. |
| **CycleDAO** | Handles cycle SQL queries, utilizing findActiveCycle() as the foundation for the daily limit engine. |
| **SavingGoalDAO** | Handles saving goal SQL queries, updating progress amounts and lifecycle statuses. |
| **UserService** | Manages authentication, secure password hashing, registration, and theme updates. |
| **WalletService** | Manages wallet lifecycles and provides dynamic balance calculation across all of a user's accounts. |

# Software Design Specification

| | |
|---|---|
| **TransactionService** | The core logic hub. Enforces the Wallet XOR Goal rule, saves ledgers, and delegates balance updates to other services. |
| **CategoryService** | Safely manages category creation and deletion without corrupting historical ledger data. |
| **CycleService** | Calculates budget consumption and outputs the Safe Daily Limit engine data. Auto-archives past cycles. |
| **SavingGoalService** | Tracks goal contributions, triggering auto-completion when targets are met. |
| **IObserver** | The subscriber contract forcing all UI controllers to implement an update() method to listen for backend changes. |
| **IObservable** | The publisher contract defining methods to add, remove, and notify observers. |
| **AppEventBus** | The central radio tower (Pub/Sub hub). Services broadcast events here; Controllers listen here to automatically refresh their UI. |
| **App** | The application entry point. Bootstraps the JavaFX stage and hands control to the SceneManager. |
| **SceneManager** | The central navigation router. Caches built screens in memory to provide instantaneous page switching. |
| **BaseController** | The parent to all screens. Automatically connects child controllers to the EventBus and provides the logged-in user context. |
| **LoginController** | Authenticates users and passes the active session down to the application upon success. |

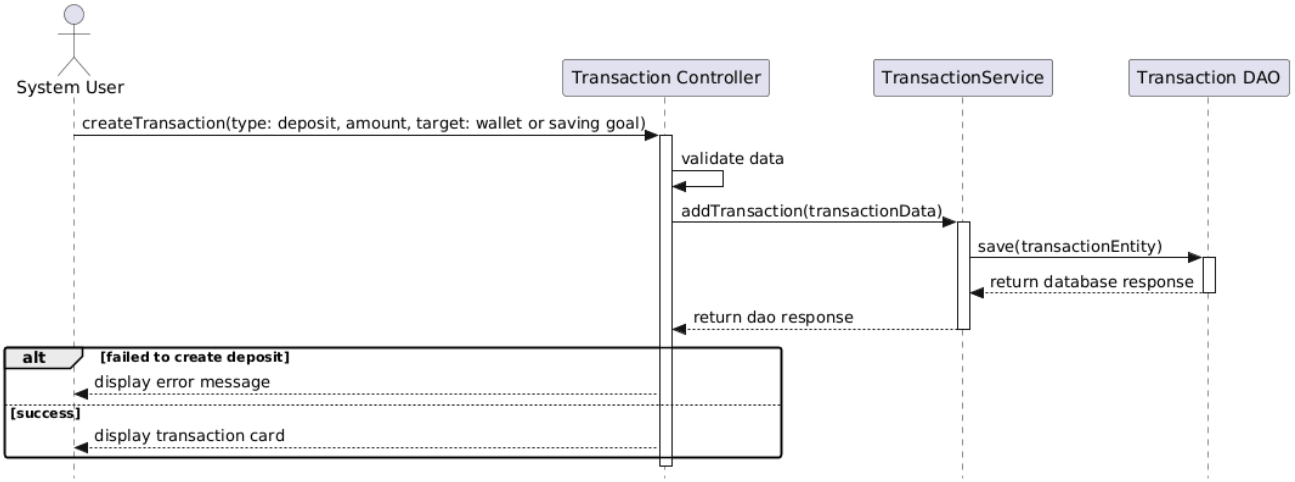
# Software Design Specification

| | |
|---|---|
| **SignUpController** | Handles new user registration with client-side validation and automatic post-creation login. |
| **DashboardController** | The primary data aggregator. Reactively updates pie charts, total balances, and the active cycle progress bar via events. |
| **TransactionController** | Manages the interactive ledger table. Features the dynamic Wallet/Goal XOR input form for strict data entry. |
| **WalletController** | Displays account balances and handles the creation of new financial containers. |
| **CategoryController** | Provides a visual color picker and type selector for generating new custom categories. |
| **CycleController** | Manages active budget periods and displays the computed Safe Daily Limit to the user. |
| **SavingGoalController** | Displays visual progress bars for financial targets. (Funding happens exclusively via the Transaction screen). |
| **SettingsController** | Handles global UI theme toggling, secure password changes, and safe session logout protocols. |

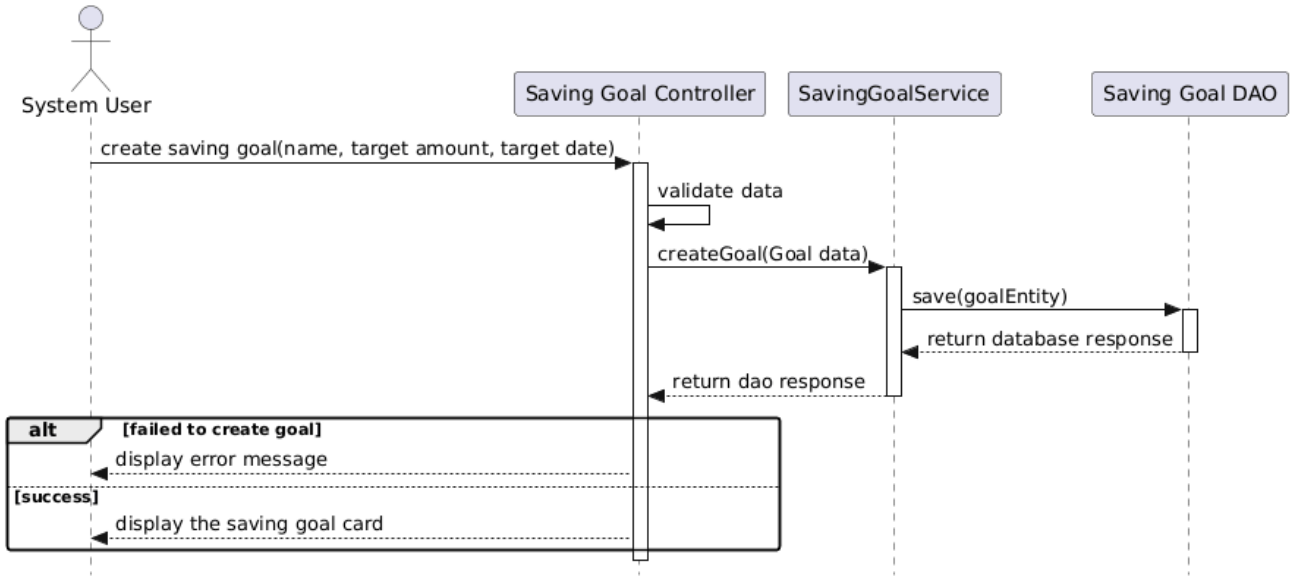
# Software Design Specification

## Sequence diagrams

### 1.Recording a Deposit

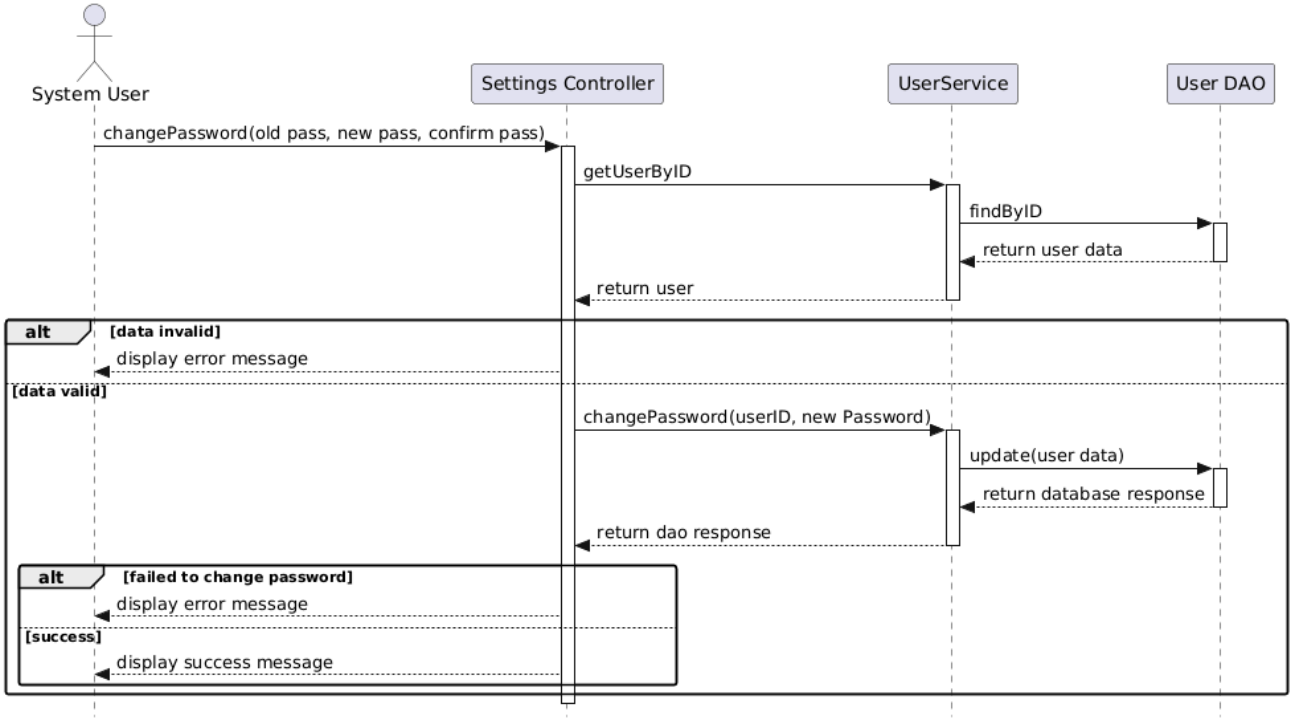



### 2.Creating Saving Goal



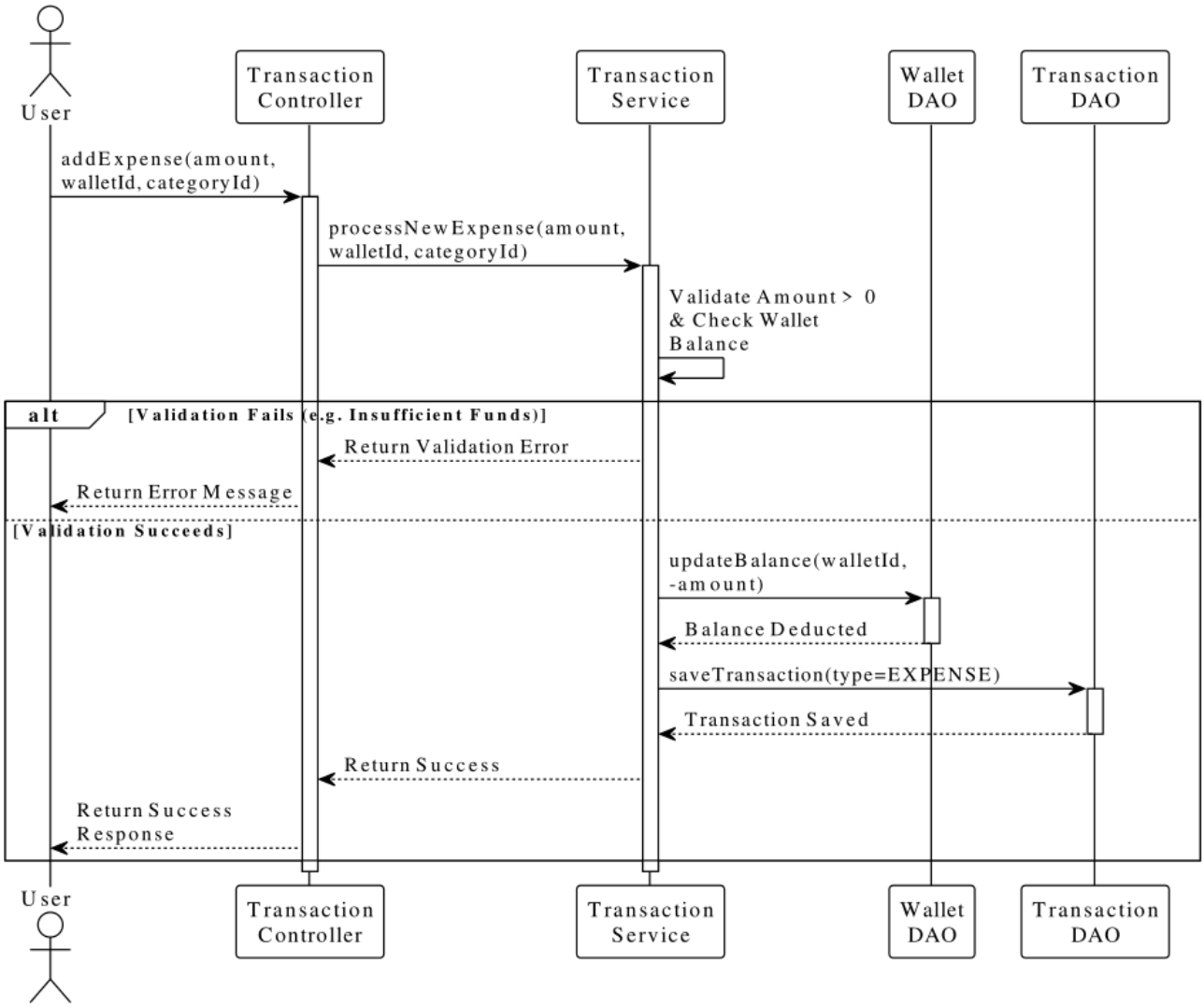
# Software Design Specification

## 3.Change Password



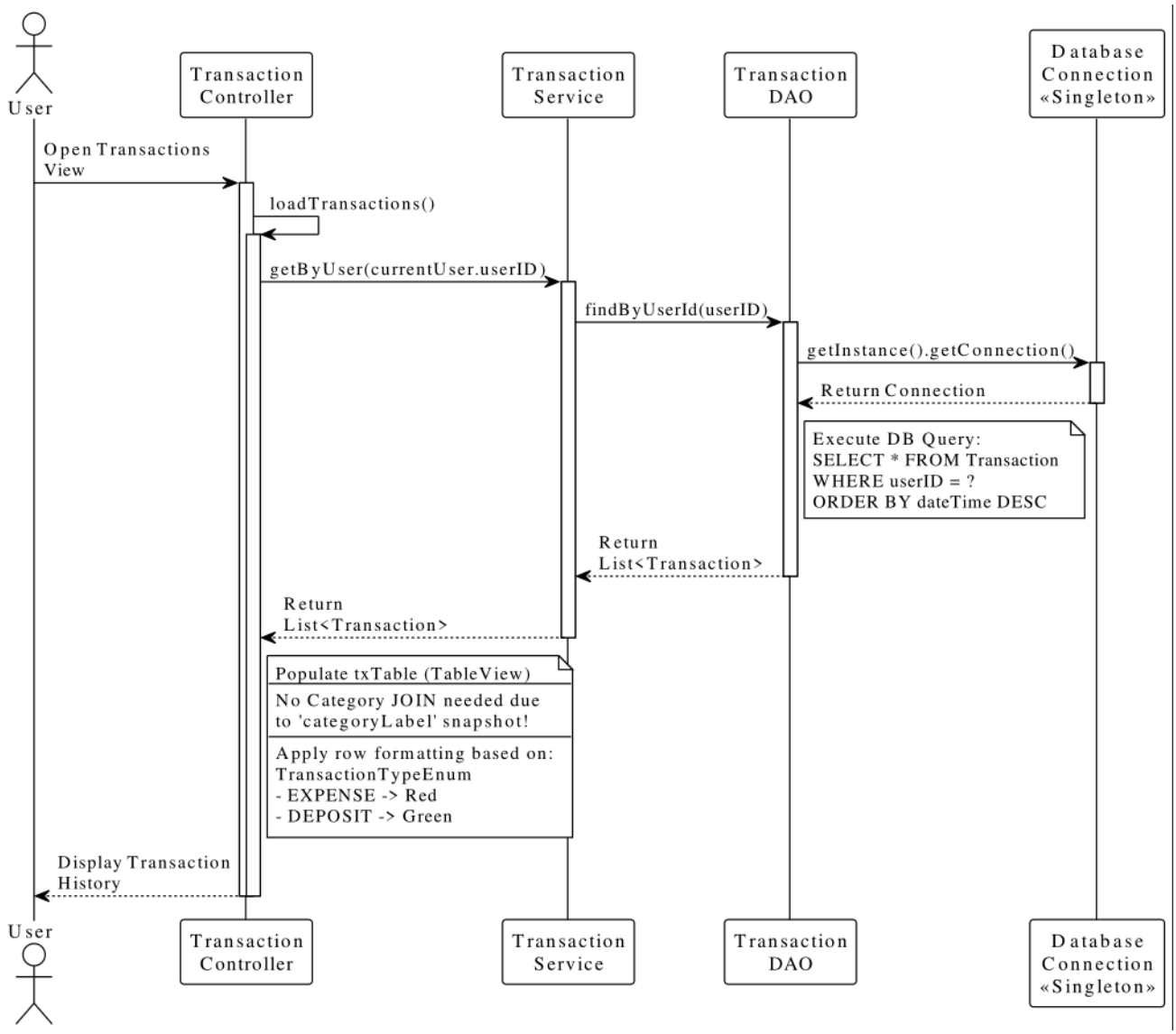
# Software Design Specification

## 4.Logging Expense



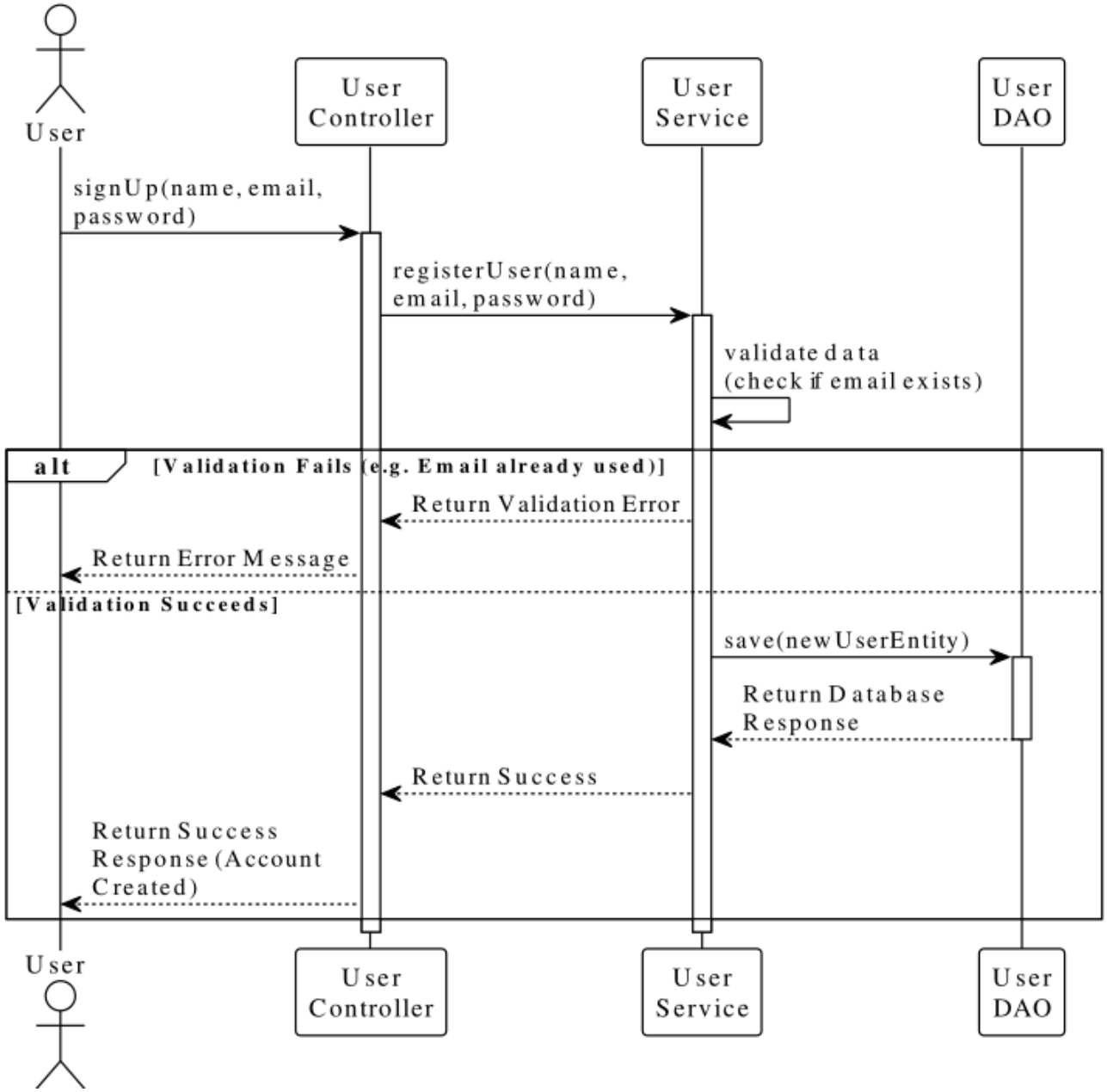
# Software Design Specification

## 5.Viewing the Financial Ledger



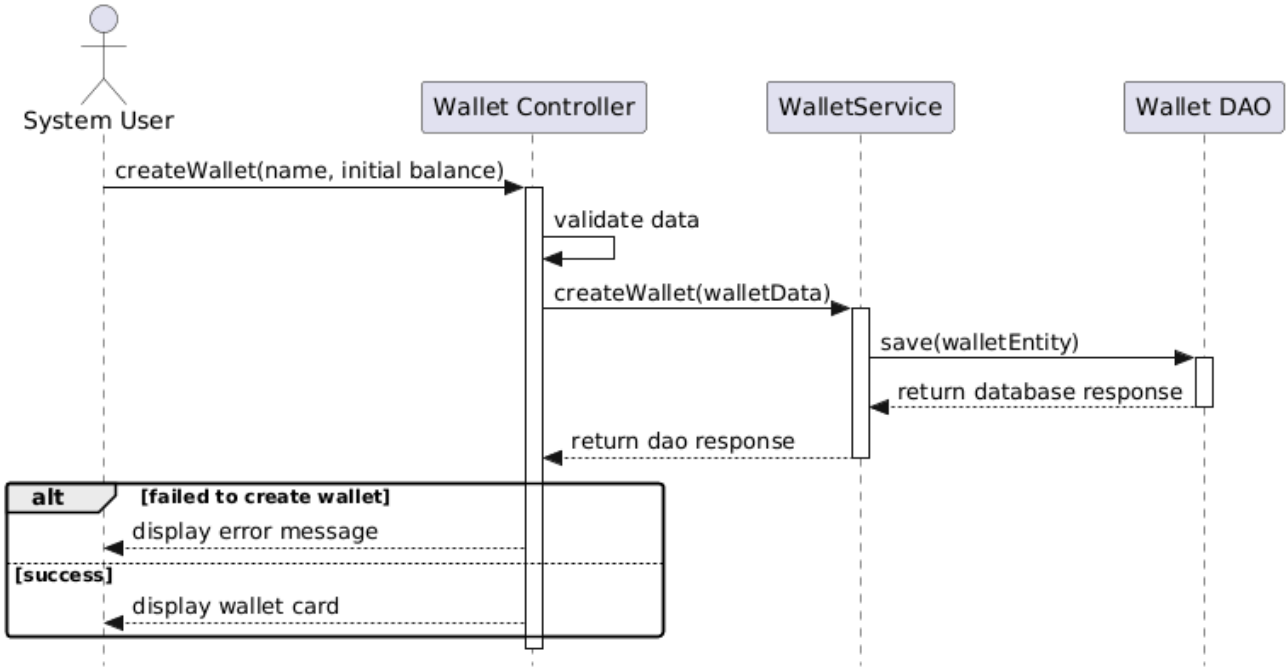
# Software Design Specification

### 6.Sign Up

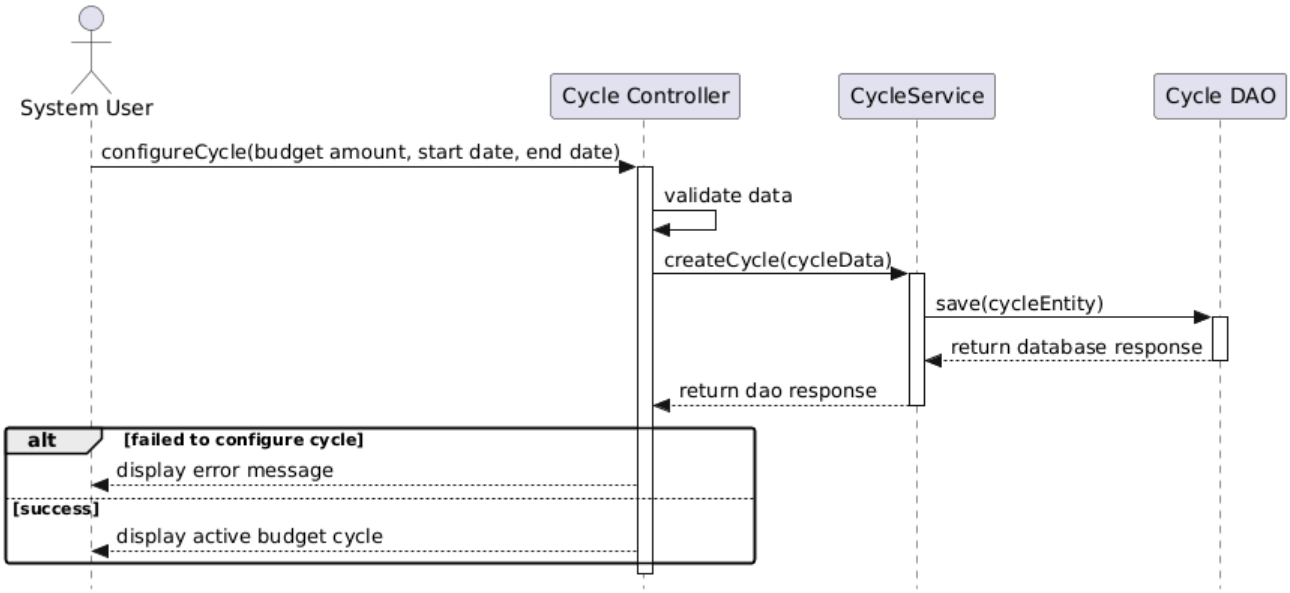


# Software Design Specification

## 7.Custom Wallet Creation

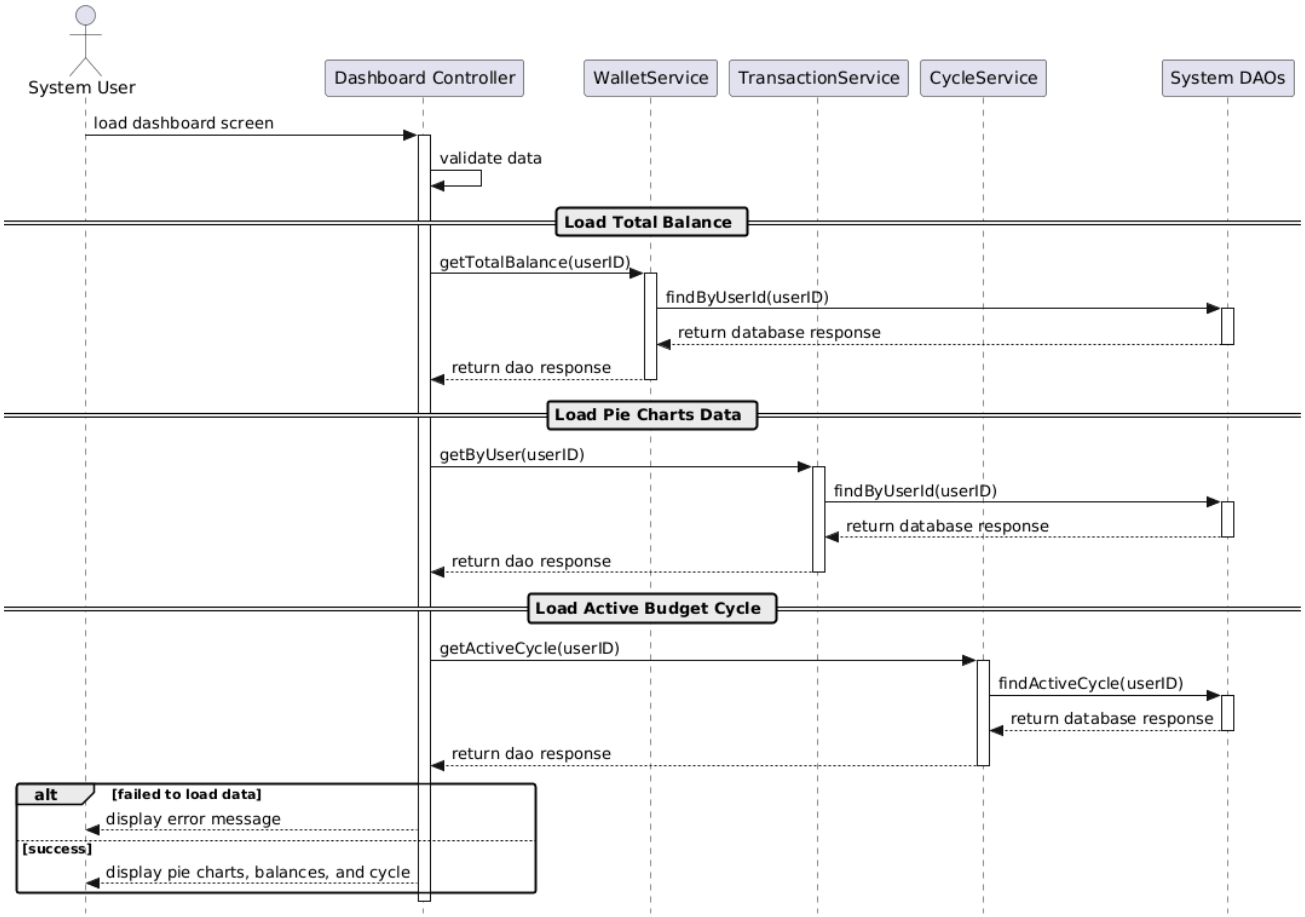



## 8.Configuring an Active Budget Cycle



# Software Design Specification

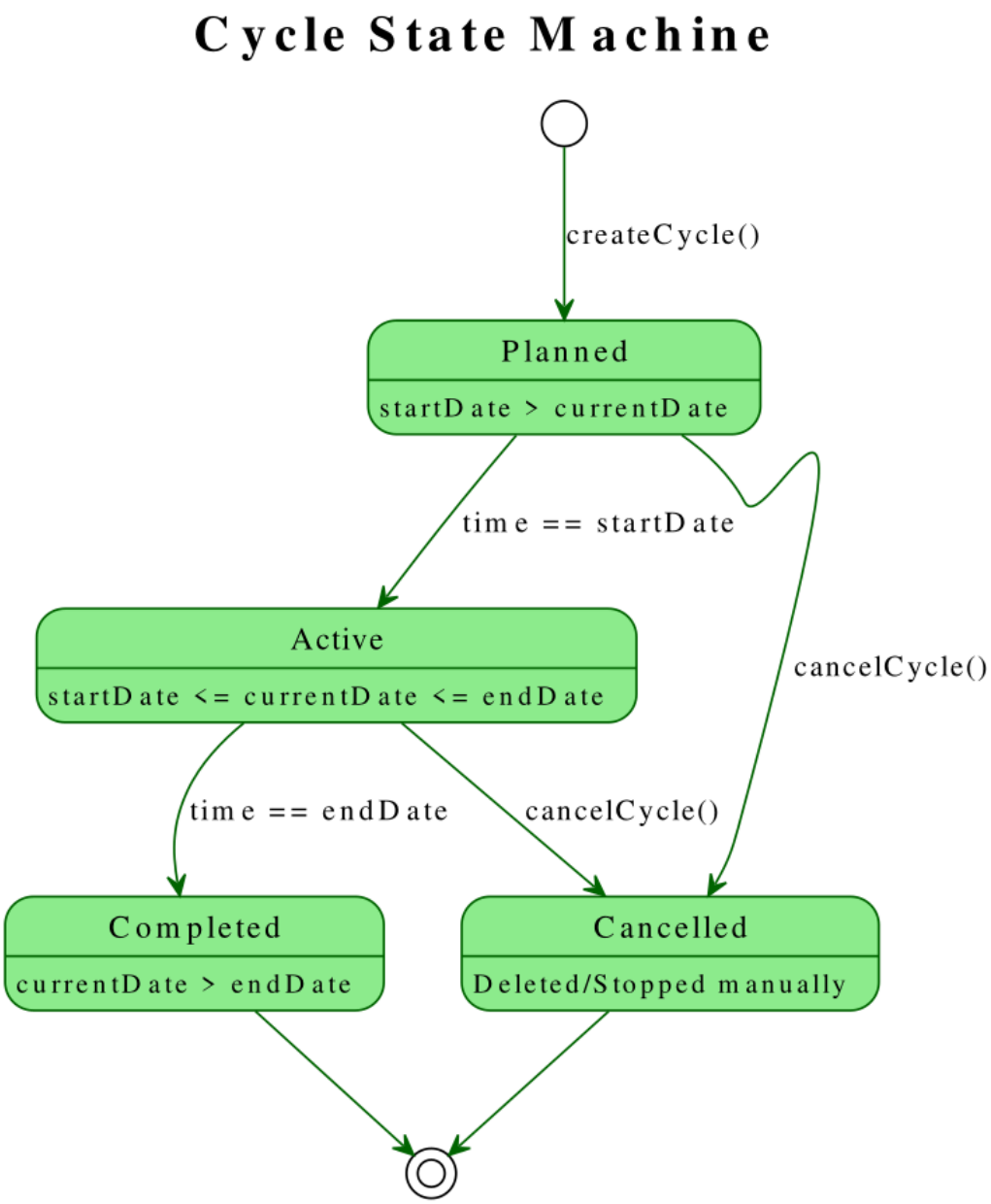
## 9.Dashboard Data Visualization



# Software Design Specification

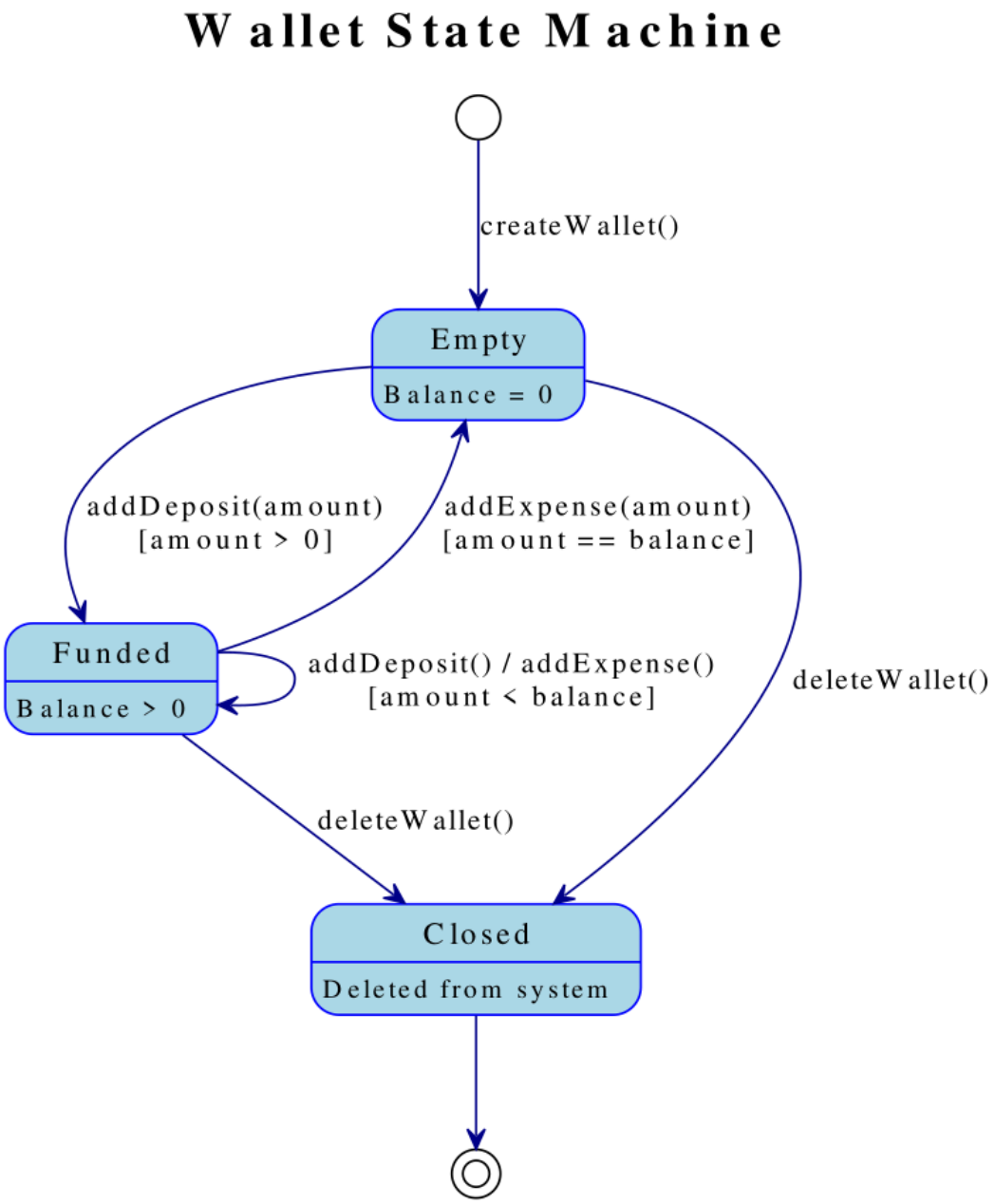
## State Diagram

### 1.Cycle State Diagram



# Software Design Specification

## 2.Wallet State Diagram



# Software Design Specification

## SOLID Principles

**Single Responsibility Principle (SRP)**

- **How it's used: The system strictly isolates responsibilities across its layers. In the Service Layer, each class manages exactly one business domain (e.g., `TransactionService` only handles transaction logic, `CycleService` handles budget math). In the GUI Layer, subclasses of `BaseController` strictly manage UI rendering and user inputs, delegating all actual data processing and business rules back to the services.**

**Open/Closed Principle (OCP)**

- **How it's used: The Database Layer is open for extension but closed for modification through the `IRepository<T>` interface. If you need to add a new data source—such as creating a `MockWalletDAO` for testing or switching from SQLite to PostgreSQL—you simply create a new class that implements `IRepository`. You do not need to modify any of the existing, functioning code in the Service Layer to support the new database.**

**Dependency Inversion Principle (DIP)**

- **How it's used: High-level modules (the Services) do not depend on low-level modules (the concrete DAOs). Instead, Service constructors require the abstraction `IRepository<T>`. For example, `WalletService` doesn't know what `WalletDAO` is; it only knows it has been given a repository that can perform standard database actions. This fully decouples the business logic from the database implementation.**

# Software Design Specification

## Design Patterns

**Singleton Pattern**

- **How it's used:** This pattern ensures that a class has only one globally accessible instance. Spenderman uses it for three critical infrastructure managers to prevent resource conflicts:
  - `DatabaseConnection`: Ensures the app doesn't open hundreds of simultaneous connections to the database, which could cause a crash or lock.
  - `SceneManager`: Coordinates UI routing centrally so multiple screens aren't competing for control of the main application window.
  - `AppEventBus`: Serves as the single source of truth for broadcasting system events.

**DAO (Data Access Object) Pattern**

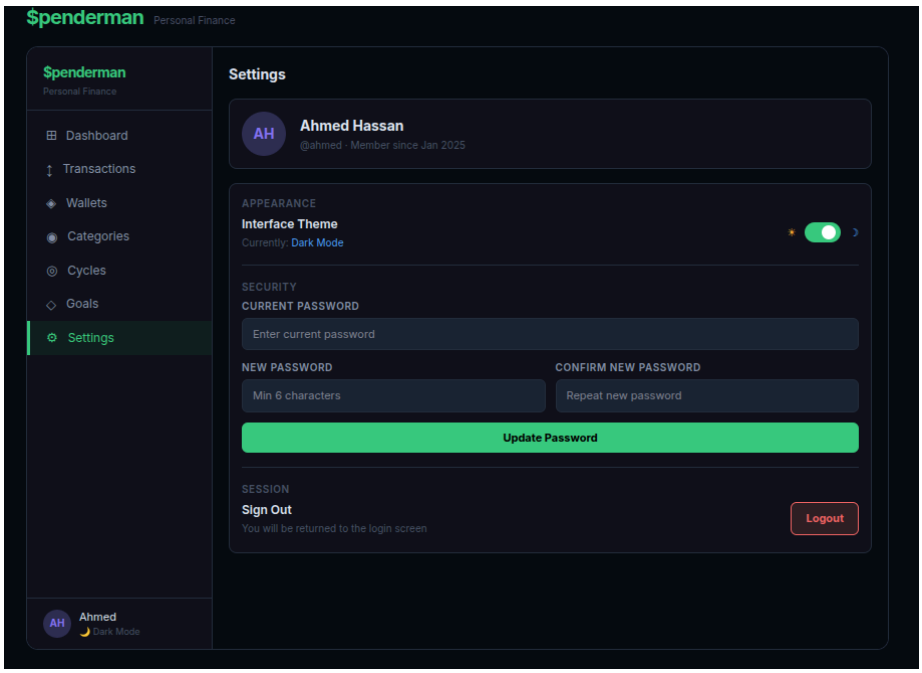
- **How it's used:** The DAO pattern abstracts and encapsulates all access to the data source. Classes like `UserDAO`, `CategoryDAO`, and `TransactionDAO` handle the raw SQL queries and data mapping. This keeps the database logic completely hidden from the Service Layer, making the system much easier to maintain, read, and test.

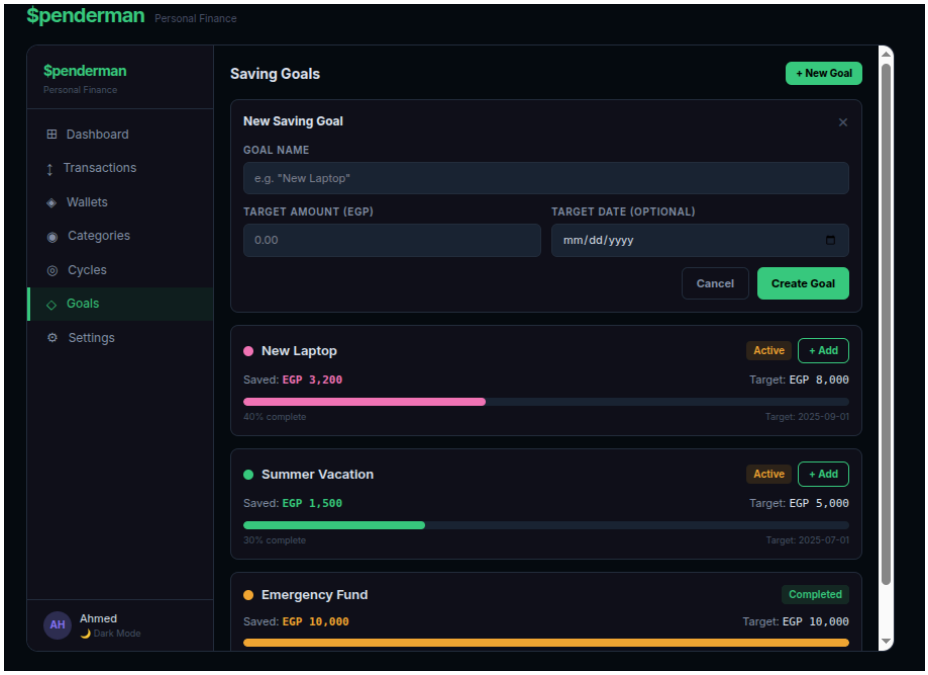
**Observer Pattern**

- **How it's used:** This pattern is used to decouple the backend business logic from the frontend UI through a publish-subscribe system. The `AppEventBus` acts as the publisher (`IObservable`). When a service updates data, it publishes an event (e.g., `"WALLET_UPDATED"`). The GUI Controllers, which implement the `IObserver` interface, listen for these events and automatically trigger a UI refresh (`refreshData()`). Because of this, the backend can update the screen without ever actually touching or knowing about the UI code.

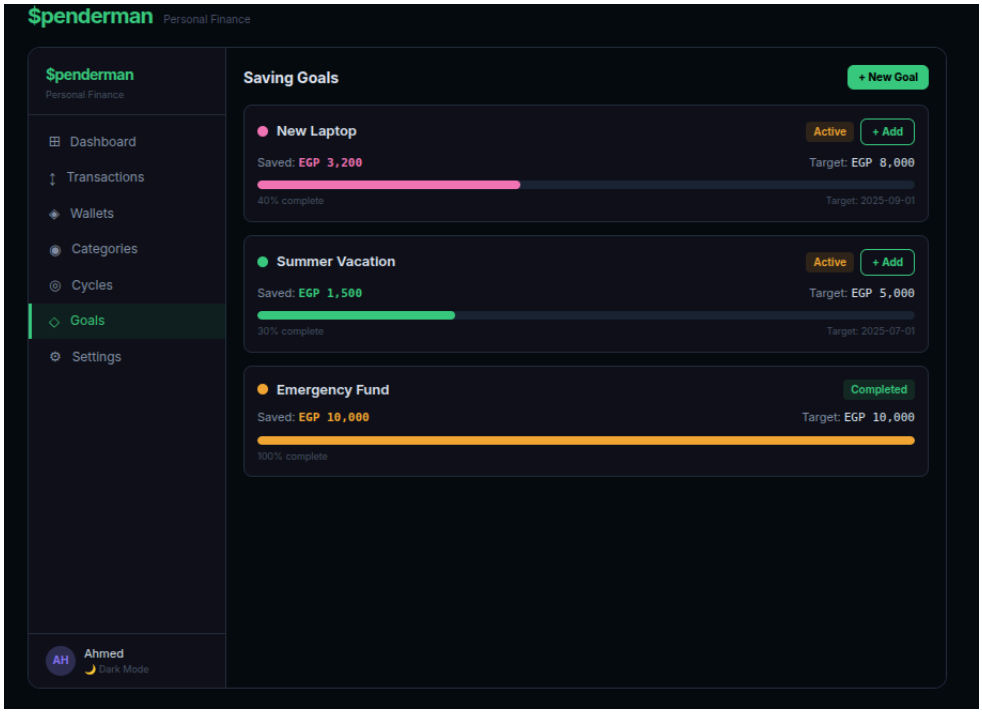
# Software Design Specification

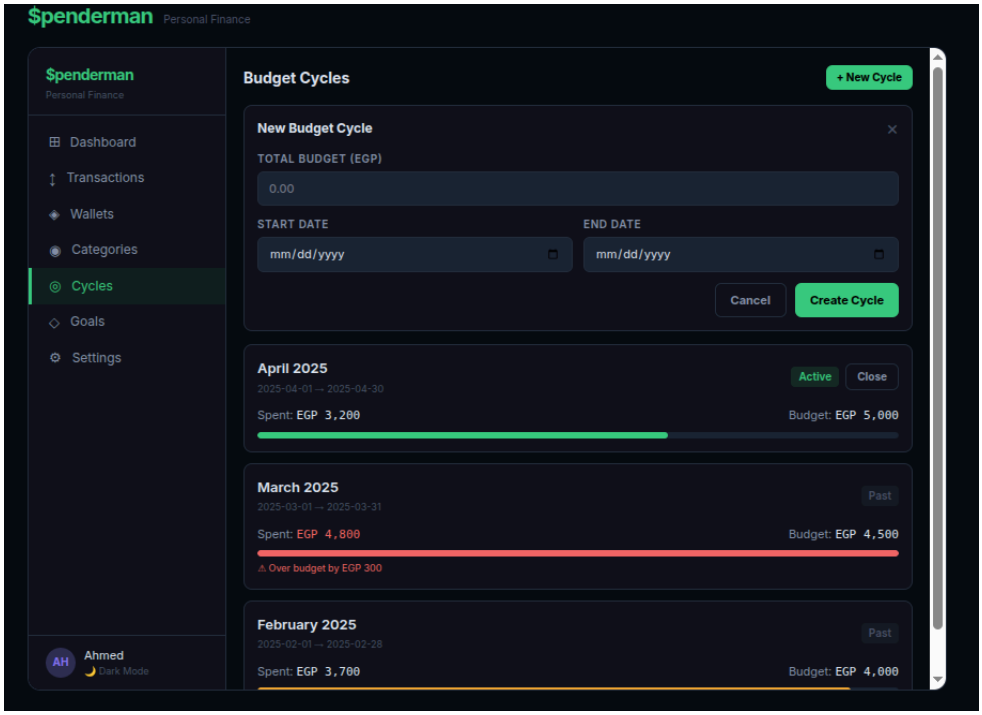
## UI snapshots



# Software Design Specification



CS251: Only three
Project: $penderman

# Software Design Specification

## Tools

PlantUML , Diagram-io , Figma

## Ownership Report

| Item | ID | Owners |
|---|---|---|
| Ahmed Amr Abdelbadea Ali Shawareb | 20240038 | ER-Diagram , Schema Diagram , UML class Diagram , 3 Sequence Diagram |
| Ahmed Mohamed Salah El-Din | 20240049 | ER-Diagram , UML class Diagram , UI design , 3 sequence Diagram |
| Mohamed Ayser Mohamed El-Hosiny | 20240476 | ER-Diagram , 2 State Diagrams , 3 Sequence Diagram , UML class Diagram |